Contents

TÀI LIỆU TỔNG HỢP ĐỒ ÁN — ĐỀ TÀI 14: MẠNG WIFI MESH KẾT NỐI DRONE & ROBOT 1
1. TỔNG QUAN ĐỀ TÀI . . . 1
2. KIẾN TRÚC HỆ THỐNG — NHÌN TOÀN CẢNH 3 LỚP . . . 2
3. VẬT TƯ, CHI PHÍ & CÔNG CỤ . . . 2
4. ROADMAP 10 TUẦN . . . 2
5. NGUYÊN LÝ MẠNG MESH & THUẬT TOÁN LÕI . . . 3
6. GIAO THỨC ĐỊNH TUYẾN MESH THAM KHẢO: B.A.T.M.A.N . . . 5
7. FIRMWARE ESP32 — KIẾN THỨC NỀN BỔ SUNG (đối chiếu kinh nghiệm PIC16F877A) . . . 5
8. MIDDLEWARE & DASHBOARD . . . 6
9. CÁC CHỈ SỐ ĐÁNH GIÁ (METRICS) — CÔNG THỨC ĐO CỤ THỂ . . . 7
10. BẢNG THUẬT NGỮ MỞ RỘNG . . . 7
11. CÂU HỎI HỘI ĐỒNG DỰ KIẾN (kèm gợi ý trả lời) . . . 8
12. HƯỚNG CẢI THIỆN & MỞ RỘNG ĐỀ TÀI . . . 8
13. HẠN CHẾ CÒN TỒN ĐỌNG (cần lường trước & nêu rõ trong báo cáo) . . . 8
14. GHI CHÚ NGUỒN . . . 10

TÀI LIỆU TỔNG HỢP ĐỒ ÁN — ĐỀ TÀI 14: MẠNG WIFI MESH KẾT NỐI DRONE & ROBOT

(SmartNetworkX & SmartIoTX)

> Tài liệu này tổng hợp toàn bộ nội dung từ 3 nguồn trong project: (1) tài liệu tổng quan đề tài, (2) lịch sử trao đổi về cơ chế self-healing và giao thức B.A.T.M.A.N, (3) tài liệu chuẩn bị kiến thức chi tiết (networking, firmware, middleware). Mục tiêu: có một bản đồ đầy đủ về hướng đi, roadmap, kiến trúc dữ liệu, kiến thức nền, và đặc biệt là các hạn chế cần lường trước khi bảo vệ đồ án.

---

1. TỔNG QUAN ĐỀ TÀI

Vấn đề thực tiễn: Hệ thống Drone/Robot truyền thống dùng WiFi hình sao (Star Topology) — bắt buộc kết nối qua 1 Router trung tâm. Router hỏng hoặc thiết bị ra xa vùng phủ sóng → toàn hệ thống tê liệt. Ở vùng thảm họa/hầm mỏ không có sóng 4G/SIM để dự phòng.

Giải pháp: Mạng Ad-hoc phi tập trung dùng ESP32 — các thiết bị tự tìm nhau, tự chuyển tiếp dữ liệu multi-hop, không cần SIM, không cần Internet, tự tạo LAN vô tuyến riêng.

Mức độ: Khá – Khó (Top Tier) vì phải giải quyết định tuyến đa chặng và phân mảnh dữ liệu ảnh.

3 tiêu chí Hội đồng Điện tử Viễn thông chấm điểm (không chấm cơ khí):

#	Tiêu chí	Ý nghĩa
1	**Throughput**	Tốc độ truyền ảnh (byte/s) suy giảm ra sao qua 2 hop, 3 hop
2	**Latency**	Độ trễ từ lúc gửi đến lúc nhận
3	**Self-healing**	Thời gian mạng tự định tuyến lại khi 1 node trung gian mất nguồn/ra khỏi vùng phủ

---

2. KIẾN TRÚC HỆ THỐNG — NHÌN TOÀN CẢNH 3 LỚP

[Lớp 1: Firmware — 4 x ESP32-CAM]
Leaf(Drone) --Mesh(multi-hop)--> Parent(Robot) --Mesh--> Root
|
UART/USB (921600 bps)
|
v
[Lớp 2: Middleware — Node.js trên PC]
serialport (đọc byte thô) --> ghép chunk theo frame_id --> socket.io (WebSocket)
|
v
[Lớp 3: Frontend — React trên trình duyệt]
Nhận Uint8Array/Base64 --> hiển thị ảnh + vẽ sơ đồ cây + thông số (Ping/Pin/FPS)

Vai trò node theo cường độ tín hiệu (RSSI): - **Root Node**: duy nhất, cắm USB trực tiếp vào PC, nơi gom toàn bộ dữ liệu. - **Parent Node**: trung gian, nhận dữ liệu từ node con và chuyển tiếp lên node cha (ví dụ: Robot). - **Leaf Node**: rìa ngoài cùng, chỉ thu thập ảnh và gửi đi (ví dụ: Drone).

---

3. VẬT TƯ, CHI PHÍ & CÔNG CỤ

Phần cứng (< 1.000.000 VNĐ): - 4x ESP32-CAM (có đế IPEX để cắm anten rời) - 4x Anten râu WiFi 2.4GHz đa hướng - 4x Mạch hạ áp Buck DC-DC (LM2596 hoặc Mini-360) — đảm bảo nguồn 5V ổn định - Pin 18650 + đế pin - Gắn lên xe RC giá rẻ hoặc cầm tay đi bộ để mô phỏng di chuyển (Mobility) — **không cần** tự chế khung Drone/Robot thật.

Phần mềm (miễn phí): - Vi điều khiển: VS Code + PlatformIO / Arduino IDE, thư viện painlessMesh - Backend: Node.js (serialport, socket.io) - Frontend: React + Tailwind CSS, host tĩnh trên Cloudflare Pages

---

4. ROADMAP 10 TUẦN

Tuần	Nội dung
1–2	Hàn mạch nguồn LM2596. Nạp painlessMesh cơ bản cho 3 node — tự tìm nhau, gửi text qua lại.
3–5	*(Khó nhất)* Viết thuật toán cắt ảnh JPEG (OV2640) thành khối 1024 byte, truyền qua Mesh, ghép lại ở Root. Chốt Motion JPEG 2–5 FPS do giới hạn băng thông.
6–7	Dựng Backend Node.js đọc Serial. Dựng React/Tailwind hứng ảnh qua WebSocket, vẽ sơ đồ Topology.
8–9	Thực nghiệm ngoài trời: ép mạng đứt kết nối để đo self-healing. Ghi log RTT, Throughput.
10	Tổng hợp số liệu, vẽ biểu đồ suy hao sóng, hoàn thiện quyển đồ án.

Gợi ý học trước từng mốc (bổ sung theo nền tảng đã có — quen PIC16F877A, chưa thực hành mạng)

- **Trước Tuần 1:** Đọc nhanh “FreeRTOS basics” (tài liệu Espressif) trước khi vào painlessMesh, để hiểu vì sao thư viện chạy “ngầm” song song với code của bạn (khác hẳn code tuần tự của PIC).

- **Trước Tuần 3:** Tự tay dựng ví dụ đơn giản — 2 ESP32 gửi qua lại chuỗi text dài > 1024 byte, tự code phân mảnh trước khi ghép logic camera vào (tránh học 2 cái khó cùng lúc: camera + fragmentation).
- **Trước Tuần 6:** Thử nghiệm nhỏ so sánh gửi ảnh test qua Base64 và qua Blob nhị phân trên cùng 1 mạng, đo chênh lệch độ trễ — có số liệu thật để trả lời câu hỏi hội đồng về Base64 vs Binary.
- **Trước Tuần 8:** Chuẩn bị kịch bản đo self-healing rõ ràng, lặp lại được (cố định vị trí 3 node, tắt nguồn node giữa tại thời điểm đánh dấu, đo bằng log timestamp của callback) — tránh đo "cảm tính bằng mắt".

---

5. NGUYÊN LÝ MẠNG MESH & THUẬT TOÁN LÕI

5.1. Mesh, Bus, hay Linear/Daisy-chain?

Câu hỏi ban đầu đặt ra: nếu 1 node ở giữa "chết" thì các node ngoài có chết theo không, và đây có phải mạng Bus không?

- **Mạng Bus** dùng chung 1 trục truyền cứng — đứt là chết toàn bộ phía sau. **Đề tài này không phải Bus.**
- Nếu các drone chỉ nối tiếp nhau theo 1 hàng, đó thực chất là **mạng Linear (đường thẳng) / Daisy-chain — một dạng suy biến của Mesh**, không phải Mesh thực thụ.
- **Về vật lý:** node ngoài cùng **không** chết theo khi node giữa mất nguồn.
- **Về mạng:** nếu node chết là "cây cầu" duy nhất → các node ngoài bị cô lập (**orphan network**), mất đường về Root. Ngược lại, nếu mật độ node đủ dày (Mesh thực thụ), các node tự tìm đường vòng qua node khác (**self-healing**).

5.2. Vì sao Multi-hop tăng "phạm vi" chứ không tăng "tốc độ"

Một ESP32 phát WiFi công suất thấp chỉ phủ sóng vài chục mét. Nếu Drone bay xa hơn, nó mất kết nối trực tiếp với Root. Multi-hop giải quyết bằng cách: Drone không cần nói chuyện trực tiếp với Root — chỉ cần nằm trong tầm sóng của 1 node khác (ví dụ Robot), node đó "tiếp sức" giúp.

Cái giá phải trả: mỗi hop tốn thêm thời gian xử lý + độ trễ truyền lại → throughput thực tế giảm dần theo số hop. Đây chính là 1 trong 3 chỉ số hội đồng chấm.

5.3. Cơ chế Self-healing (Roaming/Handover)

Tương tự khái niệm roaming khi điện thoại di chuyển giữa các trạm BTS:

1. Mỗi node liên tục đo **RSSI** từ các "hàng xóm" xung quanh.
2. Nếu RSSI với node cha hiện tại tụt dưới ngưỡng (hoặc mất hẳn) → node con hiểu đường cũ không còn tốt.
3. Node con quét lại danh sách hàng xóm, chọn node có RSSI mạnh nhất trong số đang có đường về Root → "chuyển vùng".
4. Toàn bộ chạy theo **timer lặp** — không có sự kiện tức thời báo "mất kết nối", node kiểm tra định kỳ (vài giây) trạng thái các liên kết xung quanh.

Trade-off quan trọng cần đo thực nghiệm: timer càng ngắn → mạng phục hồi càng nhanh, nhưng tốn nhiều tài nguyên xử lý + băng thông cho heartbeat hơn.

5.4. Thuật toán phân mảnh dữ liệu (Data Fragmentation)

Giới hạn WiFi (MTU) không cho phép gửi nguyên 1 ảnh ~30KB → phải chia nhỏ. Mỗi chunk mang Header gồm: frame_id (ID ảnh), chunk_id (vị trí mảnh), total_chunks (tổng số mảnh). Bên nhận (Root) cấp phát buffer, hứng đủ mảnh cùng frame_id rồi mới xuất ra.

Liên hệ lý thuyết OSI/TCP-IP: đây chính là khái niệm **Fragmentation ở tầng Network (Layer 3)**. Điểm khác biệt: IP fragmentation là cơ chế tự động, trong suốt ở tầng hệ điều hành mạng; còn ở đồ án này, vì code ở **tầng ứng dụng (Layer 7)** trên vi điều khiển, phải tự viết logic phân mảnh bằng tay — đó chính là ý nghĩa của ImageChunkPacket. Nêu được sự khác biệt này trong báo cáo cho thấy hiểu bản chất, không chỉ copy code.

Code lõi (ESP32-CAM):

```
#define CHUNK_SIZE 1024

// Cấu trúc Header của mỗi mảnh dữ liệu
#pragma pack(push, 1)
typedef struct {
    uint16_t frame_id;
    uint16_t chunk_id;
    uint16_t total_chunks;
    uint16_t data_len;
    uint8_t  payload[1024];
} ImageChunkPacket;
#pragma pack(pop)

void send_frame_over_mesh() {
    static uint16_t current_frame_id = 0;
    camera_fb_t *fb = esp_camera_fb_get(); // Chụp ảnh
    if (!fb) return;

    uint32_t total_len = fb->len;
    uint16_t total_chunks = (total_len + CHUNK_SIZE - 1) / CHUNK_SIZE;
    current_frame_id++;

    ImageChunkPacket packet;
    packet.frame_id = current_frame_id;
    packet.total_chunks = total_chunks;

    for (uint16_t i = 0; i < total_chunks; i++) {
        uint32_t offset = i * CHUNK_SIZE;
        uint16_t bytes_to_send = (total_len - offset < CHUNK_SIZE) ? (total_len - offset) : CHUNK_SIZE;

        packet.chunk_id = i;
        packet.data_len = bytes_to_send;
        memcpy(packet.payload, fb->buf + offset, bytes_to_send);

        size_t packet_size = 8 + bytes_to_send;
        // ... Lệnh gửi đi qua painlessMesh / ESP-MESH ...

        vTaskDelay(pdMS_TO_TICKS(5)); // Chống tràn bộ đệm vô tuyến
    }
    esp_camera_fb_return(fb); // Giải phóng RAM
}
```

5.5. Bảng so sánh MTU giữa các công nghệ

Công nghệ	MTU / gói tối đa	Vì sao giới hạn
WiFi (802.11)	~1500 byte	Kế thừa chuẩn Ethernet để tương thích mạng có dây

Công nghệ	MTU / gói tối đa	Vì sao giới hạn
4G/5G	~1428–1500 byte	Trừ hao overhead header đóng gói (tunneling)
Bluetooth	~23–512 byte	Ưu tiên tiết kiệm năng lượng, giảm rủi ro lỗi truyền

Ý nghĩa với đồ án: painlessMesh chạy trên WiFi nên gói tối đa lý thuyết gần 1500 byte, nhưng đề bài chủ động chọn chunk **1024 byte** — nhỏ hơn hẳn giới hạn — để: (1) chừa chỗ cho header riêng của painlessMesh cộng thêm ngoài payload, (2) giảm rủi ro mất gói khi sóng yếu (gói nhỏ → xác suất nhiễu làm hỏng cả gói thấp hơn).

---

6. GIAO THỨC ĐỊNH TUYẾN MESH THAM KHẢO: B.A.T.M.A.N

(Dùng để hiểu lý thuyết định tuyến mesh chuyên sâu — chạy trên Linux, không trực tiếp chạy trên ESP32/painlessMesh, xem lưu ý ở mục 13)

Cách hoạt động: - Chạy ở **Tầng 2 (MAC Layer)**, biến mạng thành 1 Switch ảo khổng lồ. - Không cần biết toàn bộ đường đi, chỉ quan tâm: "Next-hop tốt nhất là ai?" - Phát sóng **OGM** (Originator Messages) liên tục để đánh giá chất lượng liên kết (**TQ — Transmission Quality**) và tự động cập nhật hướng truyền khi có sự cố.

Cài đặt cơ bản (batman-adv trên Linux):

```
# Bước 1: Cài công cụ quản lý
sudo apt update && sudo apt install batctl

# Bước 2: Kích hoạt module trong nhân
sudo modprobe batman-adv

# Bước 3: Chuyển card WiFi sang chế độ Ad-hoc
sudo ip link set wlan0 down
sudo iwconfig wlan0 mode ad-hoc
sudo iwconfig wlan0 essid my-mesh-network
sudo iwconfig wlan0 channel 8
sudo ip link set wlan0 mtu 1532
sudo ip link set wlan0 up

# Bước 4: Gắn card vật lý vào mạng B.A.T.M.A.N
sudo batctl if add wlan0

# Bước 5: Khởi động và gán IP cho giao diện ảo bat0
sudo ip link set up dev bat0
sudo ip addr add 192.168.1.1/24 dev bat0
```

---

7. FIRMWARE ESP32 — KIẾN THỨC NỀN BỔ SUNG (đối chiếu kinh nghiệm PIC16F877A)

7.1. ESP32 khác PIC16F877A ở đâu

Khía cạnh	PIC16F877A	ESP32
Luồng xử lý	Đơn luồng, tuần tự	Dual-core + FreeRTOS, nhiều task song song
Mô hình I/O	Polling (hỏi liên tục trạng thái)	Event-driven — đăng ký callback (onReceive, onNewConnection…), hệ thống tự gọi khi có sự kiện
RAM	Vài trăm byte	Vài trăm KB — vẫn có thể tràn nếu giữ nhiều ảnh JPEG (~30KB/ảnh) cùng lúc trong RAM

- painlessMesh tự sinh task nền xử lý mạng WiFi liên tục, chạy **độc lập** với loop() chính → code chụp/gửi ảnh và code xử lý mạng chạy **đồng thời**, không tuần tự như PIC. Cần cẩn thận khi truy cập chung tài nguyên (ví dụ buffer ảnh) từ 2 luồng khác nhau.
- Event-driven gần với khái niệm ngắt (interrupt) đã quen, nhưng là "ngắt phần mềm" ở tầng cao hơn (do RTOS lịch trình), khác ngắt phần cứng cấp thấp cấu hình trực tiếp trên thanh ghi PIC.
- Nên gọi esp_camera_fb_return() ngay sau khi gửi xong ảnh, không giữ lại "phòng hờ" — tránh tràn RAM.

7.2. Thiết kế nguồn điện — nguyên nhân lỗi khó chẩn đoán nhất

Module WiFi của ESP32 khi phát sóng (đặc biệt lúc kết nối/gửi burst dữ liệu) có thể kéo **dòng đỉnh (peak current) 400–500mA** trong khoảnh khắc ngắn, dù dòng trung bình thấp hơn nhiều. Nếu mạch hạ áp LM2596/Mini-360 hoặc pin 18650 không đáp ứng đủ dòng đỉnh → điện áp sụt tạm thời → treo/reset ESP32. Hiện tượng này **dễ bị nhầm là lỗi phần mềm** trong khi thực chất là lỗi nguồn. Nên tìm hiểu thêm tụ lọc (capacitor) đặt gần chân nguồn ESP32-CAM để giảm hiện tượng này.

---

8. MIDDLEWARE & DASHBOARD

8.1. Vì sao UART chạy ở baud rate cao (921600 bps)?

Dữ liệu ảnh (dù đã fragment) vẫn là luồng byte lớn liên tục chảy từ Root sang PC. Nếu dùng baud rate thấp (9600/115200), cổng UART sẽ trở thành **nút thắt cổ chai (bottleneck)** còn tệ hơn cả mạng mesh không dây — nghĩa là số liệu đo throughput/latency của mesh sẽ **không chính xác** vì bị giới hạn bởi UART chứ không phải bởi sóng WiFi. Cần nêu rõ điểm này trong báo cáo.

8.2. WebSocket vs MQTT — vì sao chọn WebSocket

	MQTT	WebSocket
Mô hình	Publish/Subscribe qua broker trung gian (vd Mosquitto)	Kết nối 2 chiều trực tiếp giữa 1 server và 1 client
Phù hợp khi	Nhiều thiết bị IoT phân tán, cần độ tin cậy linh hoạt, có thể qua Internet	Chỉ 1 luồng dữ liệu tập trung (Root → Dashboard), cần độ trễ thấp
Yêu cầu hạ tầng	Cần chạy thêm broker	Không cần broker

Với kiến trúc đề tài (**chỉ 1 Root, 1 middleware, 1 dashboard**), WebSocket đơn giản và hợp lý hơn để triển khai trong thời gian ngắn. Có thể đề xuất chuyển sang MQTT ở phần "hướng phát triển" khi mở rộng lên nhiều Root/nhiều dashboard.

8.3. Luồng xử lý dữ liệu ảnh nhị phân (Binary Data Pipeline)

1. Node.js đọc byte thô từ cổng Serial (serialport).
2. Ghép các chunk cùng frame_id thành buffer đầy đủ (kiểu Buffer của Node.js).
3. Gửi qua WebSocket: **Binary/Blob** (hiệu quả hơn, khuyến khích, tiết kiệm băng thông) hoặc **Base64** (dễ debug, nhúng thẳng vào <img src="...">, nhưng tốn thêm **~33% dung lượng** so với binary gốc).
4. React nhận qua socket.io: nếu Blob → URL.createObjectURL(); nếu Base64 → gán thẳng vào src.

8.4. Checklist tính năng Dashboard

- Luồng ảnh camera thời gian thực (Motion JPEG 2–5 FPS).
- Sơ đồ cây mạng (Tree topology): Root → Parent → Leaf, **cập nhật ngay** khi có sự kiện self-healing (điểm ăn điểm trực quan khi demo).
- Thông số từng drone: **Ping** (độ trễ), **Pin** (mức năng lượng — đo qua ADC nếu có), **FPS** (số khung hình/giây thực tế) — đây là bằng chứng trực quan cho 3 chỉ số throughput/latency/self-healing, thay vì chỉ nằm trong file log.

---

9. CÁC CHỈ SỐ ĐÁNH GIÁ (METRICS) — CÔNG THỨC ĐO CỤ THỂ

Chỉ số	Công thức / Cách đo
Throughput thực tế	(Tổng số byte dữ liệu hữu ích nhận được) ÷ (Thời gian nhận). Đo ở cả 1 hop, 2 hop, 3 hop để vẽ biểu đồ suy giảm.
Latency (độ trễ)	Thời điểm Root nhận đủ 1 frame ảnh − Thời điểm Leaf bắt đầu chụp ảnh đó. Cần đồng bộ thời gian tương đối giữa các node (dùng millis() cục bộ + tính chênh lệch round-trip, vì ESP32 không có đồng hồ chung tuyệt đối).
Self-healing time	Thời điểm node phát hiện mất kết nối cũ → Thời điểm node có đường đi mới hoạt động trở lại (theo dõi callback onDroppedConnection() và onNewConnection() trong painlessMesh, ghi log timestamp).

---

10. BẢNG THUẬT NGỮ MỞ RỘNG

Thuật ngữ	Giải thích
Roaming / Handover	Node chuyển từ kết nối với node cha cũ sang node cha mới khi tín hiệu cũ yếu đi
Heartbeat	Tín hiệu “còn sống” gửi định kỳ để các node biết nhau vẫn còn kết nối
FreeRTOS	Hệ điều hành thời gian thực nhỏ chạy nền trên ESP32, quản lý nhiều task song song
Task (RTOS)	Một luồng xử lý độc lập được RTOS lập lịch chạy, khác với vòng lặp loop() tuần tự
Peak current (dòng đỉnh)	Dòng điện tăng vọt trong khoảnh khắc ngắn, thường lúc module WiFi phát sóng

Thuật ngữ	Giải thích
Broker (MQTT)	Máy chủ trung gian nhận/phân phối tin nhắn giữa các thiết bị publish/subscribe
Blob	Kiểu dữ liệu nhị phân thô trong JavaScript, truyền ảnh hiệu quả hơn Base64
OGM (B.A.T.M.A.N)	Originator Message — gói tin quảng bá liên tục để đánh giá chất lượng liên kết (TQ)
Orphan network	Nhóm node bị cô lập khỏi Root khi mất "cây cầu" kết nối duy nhất
Flapping	Hiện tượng đổi node cha liên tục khi tín hiệu dao động nhẹ quanh ngưỡng self-healing

---

11. CÂU HỎI HỘI ĐỒNG DỰ KIẾN (kèm gợi ý trả lời)

1. **Vì sao chọn baud rate UART 921600 thay vì 115200?** Nếu để 115200 thì kết quả đo throughput sẽ bị giới hạn bởi chính cổng UART chứ không phản ánh đúng năng lực mạng mesh.
2. **Self-healing dựa trên timer lặp — nếu timer quá ngắn có hại gì?** Tốn băng thông cho heartbeat; có thể gây "flapping" — đổi node cha liên tục nếu tín hiệu dao động nhẹ quanh ngưỡng.
3. **Vì sao không dùng MQTT — chuẩn IoT phổ biến — mà lại chọn WebSocket?** Kiến trúc chỉ có 1 luồng dữ liệu tập trung (Root → Dashboard), không cần broker trung gian, độ trễ thấp hơn.
4. **Gửi ảnh Base64 hay Binary/Blob, chọn cái nào và đánh đổi ra sao?** Blob hiệu quả băng thông hơn (Base64 tốn thêm ~33% dung lượng); Base64 dễ debug/nhúng trực tiếp hơn.
5. **ESP32-CAM treo/reset ngẫu nhiên lúc phát WiFi mạnh — nguyên nhân phần cứng nào cần loại trừ trước khi nghi ngờ phần mềm?** Sụt áp do dòng đỉnh (peak current 400–500mA) vượt khả năng đáp ứng của mạch hạ áp/pin.

---

12. HƯỚNG CẢI THIỆN & MỞ RỘNG ĐỀ TÀI

- **Giao thức định tuyến động nâng cao:** tham khảo AODV, OLSR, B.A.T.M.A.N để hiểu sâu hơn cơ chế chọn đường ngoài mức mà painlessMesh cung cấp sẵn.
- **Tăng mật độ Node (Redundancy):** bay/di chuyển theo cụm (swarm) hoặc zig-zag để tăng liên kết chéo giữa các node, giảm nguy cơ orphan network khi có node hoặc kết nối bị mất.
- **Thuật toán Tự điều chỉnh Vị trí (Topology Control):** lập trình để node tự bù đắp khoảng trống vùng phủ sóng khi có 1 node khác rớt mạng.
- **Quản lý Năng lượng (Energy-Aware Routing):** ưu tiên chọn node còn nhiều pin làm node chuyển tiếp (parent), thay vì chỉ dựa thuần vào RSSI.
- **Chất lượng Dịch vụ (QoS):** ưu tiên băng thông cho dữ liệu điều khiển (control) hơn dữ liệu video/ảnh khi mạng suy hao mạnh.
- **Nâng cấp middleware:** đề xuất chuyển từ WebSocket sang MQTT khi mở rộng hệ thống lên nhiều Root/nhiều Dashboard cùng lúc.

---

13. HẠN CHẾ CÒN TỒN ĐỌNG (cần lường trước & nêu rõ trong báo cáo)

Đây là các điểm yếu/rủi ro kỹ thuật đã được xác định qua các lần trao đổi, nên đưa vào phần "Hạn chế của đề tài" hoặc "Hướng phát triển" của báo cáo để chủ động, tránh bị hội đồng "bắt bài":

13.1. Hạn chế về kiến trúc mạng

- **Mạng hiện tại có nguy cơ suy biến thành Linear/Daisy-chain** thay vì Mesh dày đặc thực thụ, do chỉ có 4 node. Nếu 1 node đóng vai "cây cầu" duy nhất bị mất, các node phía sau sẽ bị cô lập hoàn toàn (**orphan network**) — self-healing chỉ hoạt động tốt khi có node dự phòng nằm trong tầm phủ chéo.
- **Root Node phải cắm dây USB cố định vào PC** → Root không di động được, giới hạn kịch bản triển khai thực tế (không thể có Root "di chuyển tự do" như Drone/Robot khác).
- Multi-hop **đánh đổi phạm vi lấy tốc độ**: throughput suy giảm dần theo số hop — đây là giới hạn vật lý không thể loại bỏ hoàn toàn, chỉ có thể đo và trình bày rõ mức suy giảm.

13.2. Hạn chế của cơ chế Self-healing

- Dựa trên **timer lặp (polling)**, không phải sự kiện tức thời → luôn có độ trễ phát hiện mất kết nối, khoảng trễ này phụ thuộc chu kỳ timer.
- **Trade-off không thể tối ưu cả hai chiều cùng lúc**: timer ngắn → phục hồi nhanh nhưng tốn tài nguyên/băng thông cho heartbeat; timer dài → tiết kiệm tài nguyên nhưng phục hồi chậm.
- Nguy cơ **"flapping"** — node đổi cha liên tục nếu RSSI dao động nhẹ quanh ngưỡng, gây bất ổn định mạng nếu không có cơ chế hysteresis (ngưỡng trễ) phù hợp.

13.3. Hạn chế về băng thông & chất lượng ảnh

- Giới hạn băng thông buộc phải chốt **Motion JPEG 2–5 FPS** — đây không phải video mượt thực sự, chỉ là chuỗi ảnh tĩnh liên tiếp.
- **UART có thể trở thành bottleneck** nếu không cấu hình đúng baud rate cao (921600 bps); nếu đo đạc sai baud rate, số liệu throughput/latency sẽ phản ánh sai bản chất mạng mesh không dây.
- **Base64 tốn thêm ~33% dung lượng** so với truyền Binary/Blob — cần cân nhắc kỹ khi chọn phương án truyền ảnh qua WebSocket.

13.4. Hạn chế phần cứng & firmware

- **RAM vẫn là tài nguyên khan hiếm** dù ESP32 có nhiều hơn PIC: ảnh JPEG ~30KB cộng buffer WiFi stack có thể gây tràn bộ nhớ nếu giữ nhiều ảnh cùng lúc.
- **Race condition tiềm ẩn**: code chụp/gửi ảnh và task nền xử lý mạng của painlessMesh chạy đồng thời trên FreeRTOS (khác hẳn mô hình tuần tự của PIC16F877A) → cần cẩn trọng khi truy cập chung tài nguyên (buffer ảnh).
- **Thiết kế nguồn điện dễ gây lỗi khó chẩn đoán nhất**: dòng đỉnh 400–500mA lúc phát WiFi có thể làm sụt áp, treo/reset ESP32 — hiện tượng này dễ bị nhầm là lỗi phần mềm trong khi bản chất là lỗi nguồn.
- **Không có đồng hồ chung tuyệt đối giữa các node** → đo latency có sai số hệ thống, phải bù bằng tính round-trip từ millis() cục bộ, không thể đạt độ chính xác tuyệt đối.

13.5. Hạn chế về phạm vi & tính đại diện của đồ án

- Mô phỏng bằng **xe RC/cầm tay** thay vì Drone/Robot thật → kết quả đo (RSSI, self-healing) có thể khác trong điều kiện thực tế có gió, rung động, hoặc tốc độ di chuyển cao hơn.
- **Thời gian đồ án ngắn (10–15 tuần)** không đủ để tự cài đặt giao thức định tuyến phức tạp thực thụ (AODV/OLSR/B.A.T.M.A.N) trên ESP32 — chỉ dùng được thư viện có sẵn (painlessMesh) với thuật toán chọn cha đơn giản dựa RSSI, không đạt độ tinh vi của các giao thức mesh chuyên dụng.
- **B.A.T.M.A.N chạy ở tầng Linux/card WiFi hỗ trợ ad-hoc**, không thể chạy trực tiếp trên ESP32/painlessMesh (painlessMesh dùng giao thức mesh riêng của nó) — cần nói rõ trong báo cáo rằng B.A.T.M.A.N chỉ mang tính **tham khảo lý thuyết định tuyến mesh nói chung**, không phải công nghệ được cài đặt trực tiếp trong đồ án, để tránh gây hiểu nhầm khi hội đồng hỏi.

---

14. GHI CHÚ NGUỒN

Tài liệu này tổng hợp từ 3 nguồn trong project: 1. gemini-code-1787323211193.md — Tổng quan đề tài, vật tư, roadmap, code lõi phân mảnh ảnh. 2. Tong_Hop_Mang_Mesh_Drone.md — Trao đổi về bản chất mạng Mesh/Linear, self-healing, và giao thức B.A.T.M.A.N. 3. chuan_bi_kien_thuc_do_an_wifi_mesh.pdf — Tài liệu chuẩn bị kiến thức chi tiết: lý thuyết mạng (multi-hop, self-healing, MTU/fragmentation), firmware ESP32 (đối chiếu PIC16F877A, thiết kế nguồn), middleware/dashboard (UART, WebSocket vs MQTT, binary pipeline), metrics, thuật ngữ, và câu hỏi hội đồng mở rộng.